Splash Screen

IMPLEMENTATION DOCUMENTATION

The Ripple Effect

Prepared by :

Juanette Rozaan Viljoen - 224145312
Tinotenda Mhedziso - 227284240
Line Redpath - 224350536

Table of Contents

Introduction

This document details the technical implementation of *SplashScreen*. The primary purpose is to outline the key development tools selected, present complex or critical extracts of our codebase, provide references for third-party code utilized, and offer a reflective summary of the major technical challenges encountered and overcome during our development process. The focal point will be the core concepts and unique solutions employed within the Android environment.

1.1 Choice of Tools

SplashScreen has been built as a native Android mobile application based on the criteria outlined in our business case. This platform choice was mandatory because the application's core functionality is centered on providing maximum accessibility to our target market: individual pool owners and pool service providers. Mobile applications are best suited for:

Immediate Utility at the Poolside: Mobile devices are widely used and portable, enabling users to log chemical readings, update maintenance records, and check water restrictions directly at the point of action: the swimming pool. Building a desktop or web app would lack the instant access and offline capabilities often needed in this environment.

Service Provider Mobility: For maintenance businesses, the solution must support staff who are constantly mobile. Native GPS capabilities such as localized weather, alerts and instant push notifications for scheduling are superior on a mobile platform compared to web or desktop environments.

Hardware Integration: Native development allows direct, efficient access to hardware features like the camera and GPS, which are essential for core feature delivery. We plan on extending the functionality after the MVP submission to include future image analysis of test strips using a machine learning model.

The following specific development tools were selected to facilitate the creation of this high-performance, native Android application, based on criteria such as platform compatibility, robust community support, and ease of integration.

1.1.1 Android Studio

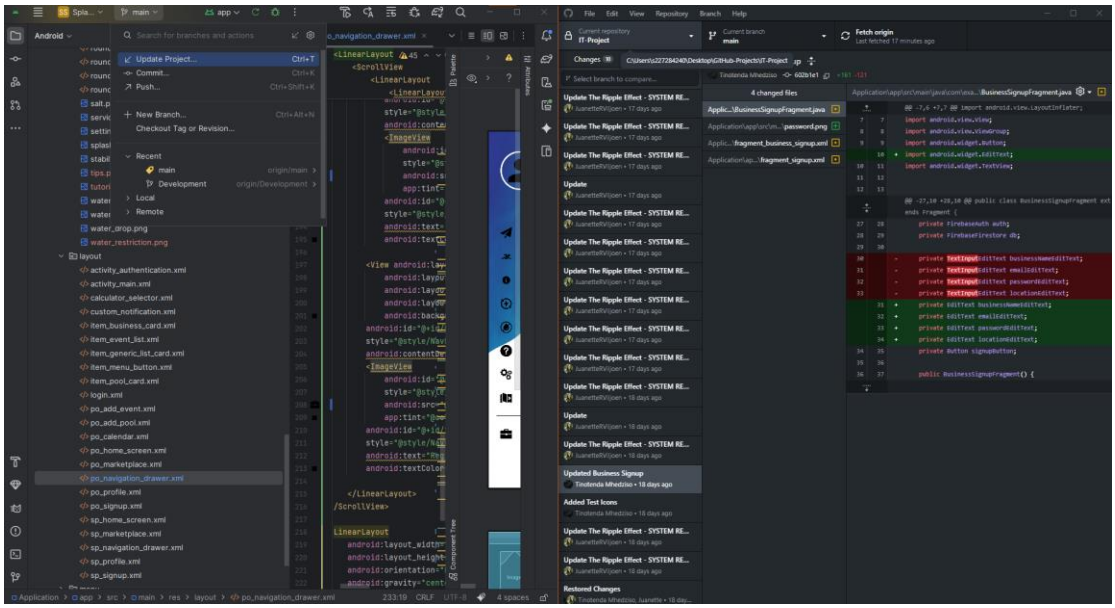

Figure 1.1 Android Studio's embedded version control support

Android Studio was chosen as the primary Integrated Development Environment (IDE) because it's the official and most comprehensive tool for native Android development. It provides essential features such as the Android SDK, an integrated emulator, an intuitive XML layout editor, and robust debugging tools, the Layout Inspector and Profiler. Android Studio's native support for the Gradle build system and version control streamlines the entire development workflow for a high-performance, platform-optimized application. **Figure 1.1** shows the direct version control capabilities embedded within Android Studio, a feature that has been crucial within our development journey.

For an app like SplashScreen, building native mobile app was a critical and important decision. It gives us a direct access to the device's hardware and the latest Android features without the performance overhead that can come from a cross-platform abstraction layer, like in React Native or Flutter. This means faster app load times, a more responsive user interface and a smaller initial app size, all key factors for a great user experience. For android development, we weren't as limited to device capabilities compared to iOS app development.

We decided to go for the traditional stack of Java and XML. While modern alternatives like Kotlin and Jetpack Compose offer powerful features, Java's long-standing stability and immense library support provided us a reliable foundation. This decision was also met after consulting some tutorials (Vogella, 2023). The visual XML Layout Editor was also incredibly valuable for rapidly designing and iterating on *SplashScreen's* complex user interfaces, from the custom navigation drawer to the detailed calculator screens, developing our designs, which some were based on UiLover, was made easier (UiLover, n.d.). Given the project's timeline, leveraging our existing proficiency in this stack was a pragmatic choice. It allowed us to focus our efforts on implementing robust features and adopting modern architectural patterns, like the Shared View Model, rather than learning an entirely new language and UI framework from the ground up (Android Developers, 2025).

1.1.2 Firebase

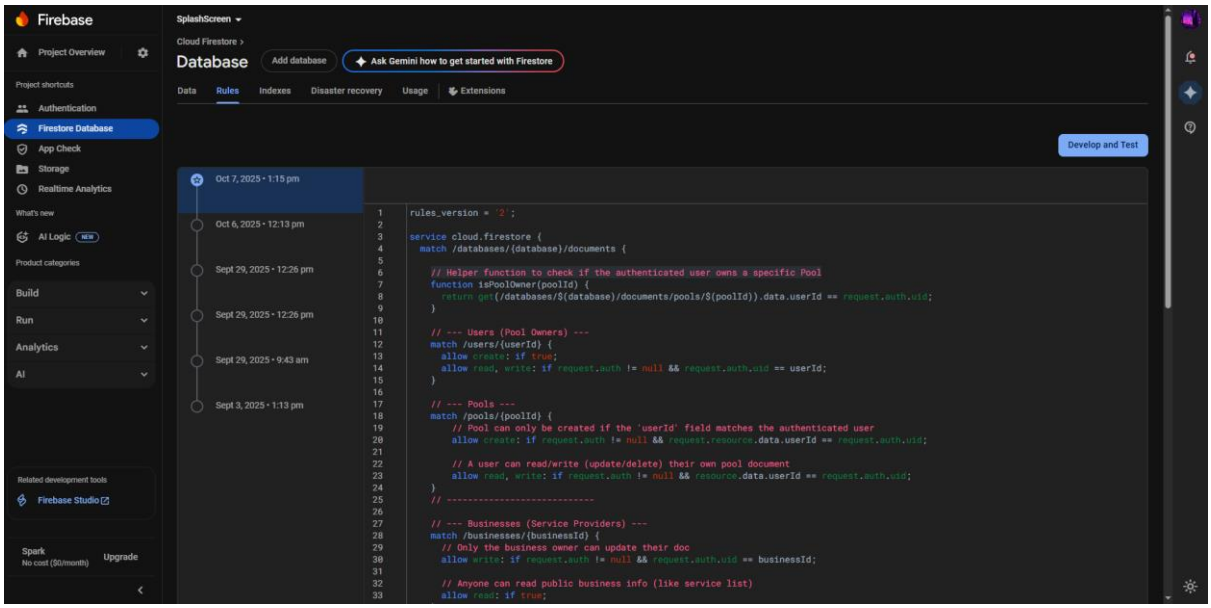

Figure 1.2 Custom Firestore Security Rules enforcing user-level and business-level data ownership for the SplashScreen database

For our backend we selected Firebase, a platform developed by Google, for its comprehensive suite of backend services. Its inclusion avoids the need to build and maintain a custom server infrastructure, significantly accelerating our deployment. The key services we utilized include Firebase Authentication for secure user login/registration and Cloud Firestore for efficient and scalable data storage.

We chose this non-relational database platform primarily for its developer velocity and powerful real-time capabilities. While Cloud Firestore is NoSQL, our data structure was carefully designed to mirror relational database best practices, ensuring data integrity comparable to normalization standards. The core advantage for *SplashScreen*, however, lies in Firebase's comprehensive security and data delivery:

Authentication and Data Modularity: Firebase natively handles secure user sign-in and provides real-time data synchronization. This was critical for building features like our maintenance event management functionality, ensuring users receive immediate updates when they create, modify, or delete maintenance events or logs from any authenticated device.

Security Rules: The platform allows for precise control over data access via Firestore Security Rules. As demonstrated in **Figure 1.2**, every read and write operation is tightly coupled to the authenticated user's ID `(request.auth.uid)`, ensuring that a pool owner can only access or modify data belonging to their specific pool document.

App Integrity: To further secure the backend, we utilized Firebase App Check. This feature ensures that database queries and network requests originating from the client are only accepted if they come from a verified instance of the SplashScreen application, effectively

mitigating denial-of-service and unauthorized access attempts. Every signed version of the SplashScreen .apk we have additional configurations that will conform to this.
This robust framework ensures the application is not only fast and responsive but also highly secure and scalable as the user base grows (Firebase Developers, 2025).

1.2 Extracts of complex code

This section presents our most complex operations, as of October 2025 from the *SplashScreen* codebase.

1.2.1 Loading events and real-time management

Theoretical Anatomy

Extract: The `loadEventsForSelectedDate` Method for Real-Time Event Management

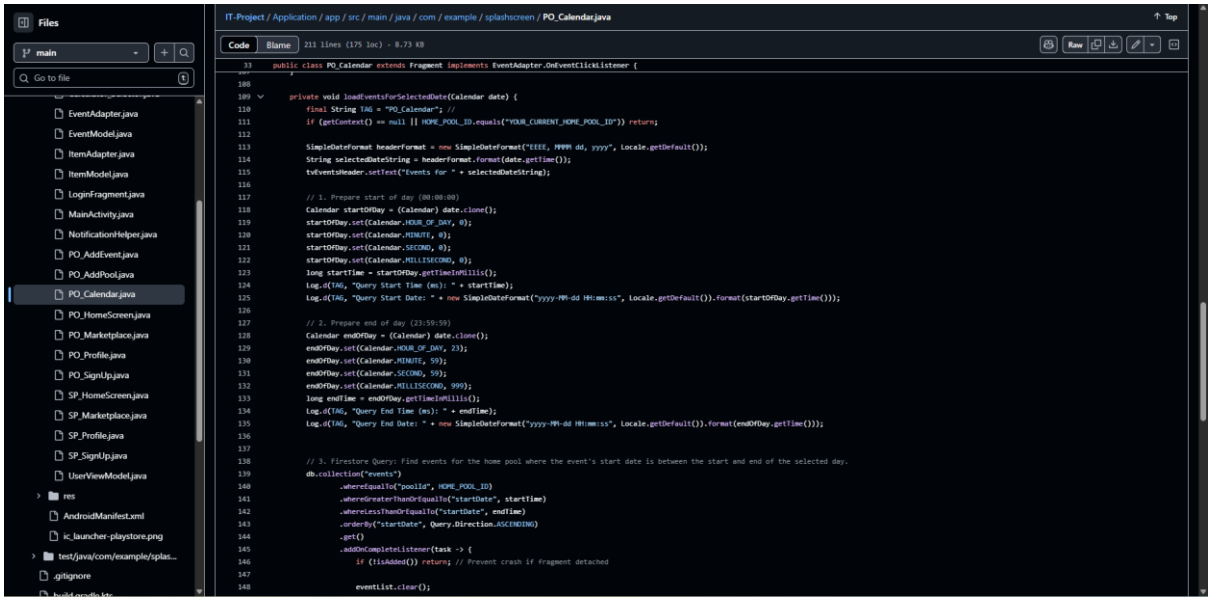

Figure 1.3 A method for real-time maintenance event management

The `loadEventsForSelectedDate` method, seen in **Figure 1.3** and located within the `PO_Calendar` Fragment, manages the entire workflow for fetching, filtering, ordering, and displaying pool maintenance events for a specific day. For understanding we have added comments in a step-based approach. Its complexity originates from the need to translate a user's date selection, “a human-readable day, into a precise, secure, and performant Firestore database query, while also ensuring the data is presented chronologically and updated asynchronously. This functionality is the backbone of our calendar and scheduling system.

The initial challenge was establishing a security architecture where events are tightly linked to a specific `poolId`, which is itself tied to the authenticated user. This was complicated further by the need to later allow service providers, who are not the pool owner, to also read

and write events for that pool, necessitating careful design of the database rules **Figure 1.2** to allow this delegated access.

Technical Anatomy

Date Range Calculation: A core technical difficulty was accurately querying a single 24-hour period. Standard date queries often fail due to time zone issues or imprecise timestamps. The method solves this by cloning the selected date and explicitly setting the time components to the millisecond:

`startOfDay` is set to 00:00:00.000
`endOfDay` is set to 23:59:59.999

This converts the calendar date into two precise Unix timestamps, startTime and endTime, for accurate range filtering in Firestore.

Complex Firestore Querying: The application needed to satisfy three simultaneous query constraints:

- `whereEqualTo("poolId", HOME_POOL_ID)`: Enforces the architectural rule that events are tied to the active pool (not just the user).
- `whereGreaterThanOrEqualTo("startDate", startTime)`
- `whereLessThanOrEqualTo("startDate", endTime)`: Filters the events to exactly the selected 24-hour period.
- `orderBy("startDate", Query.Direction.ASCENDING)`: Crucially sorts the events chronologically, ensuring the schedule appears correctly on the user interface.

Backend Optimization (Indexing): Combining a range filter: `(whereGreaterThanOrEqualTo/whereLessThanOrEqualTo)` with an equality filter `(whereEqualTo)` and a sort or order by makes this a composite query. This required the manual creation of a Custom Index in the Firebase console, otherwise, the query would fail. This backend step was essential for maintaining query speed and efficiency.

Asynchronous Data Handling: The `get()` function executes asynchronously, meaning the main app thread is not blocked while fetching data. The result is handled within the .addOnCompleteListener callback. This design ensures the app remains responsive, a key requirement of modern Android development. The retrieved JSON data is automatically mapped to the local `EventModel.Java` object using Firestore's built-in object mapping capabilities.

1.2.2 Custom navigation logic

Extract: The `setupDrawer` and `onNavigationItemSelected` Methods for System navigation

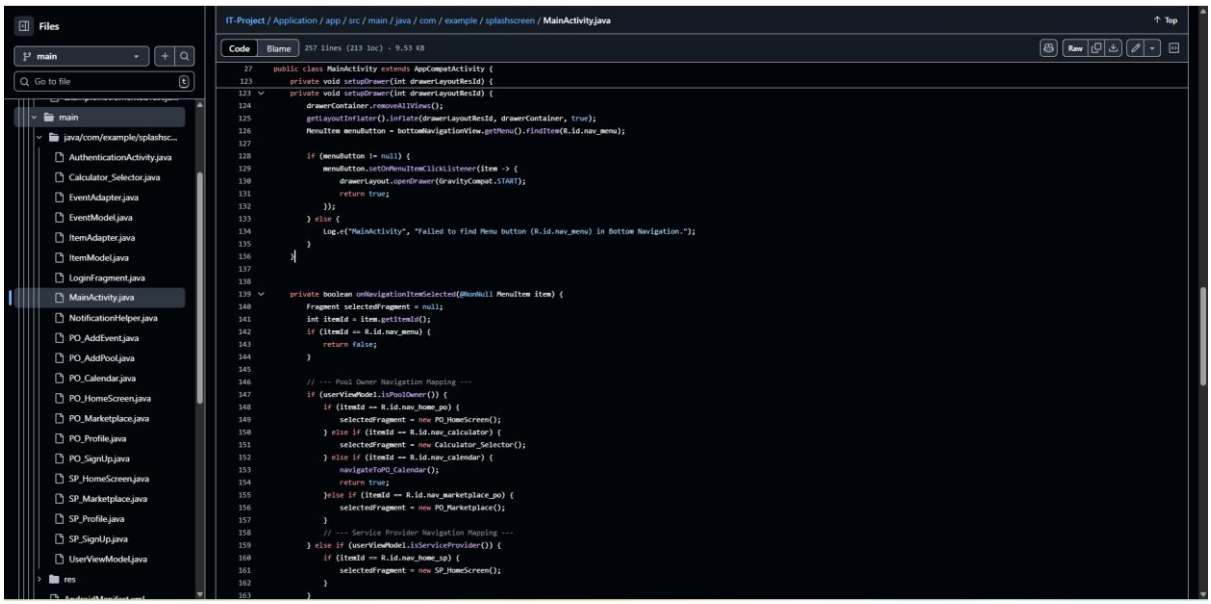

Figure 1.4 A code snippet for the navigation logic

Theoretical Anatomy

The navigation structure for *SplashScreen* is inherently complex because it must support two distinct user roles: Pool Owner and Service Provider, each requiring a unique set of menu items and fragment destinations, all triggered from a persistent button in the main activity, the menu button. Our initial attempts at implementing the custom drawer were highly inefficient, involving us manually adding the drawer layout to multiple screens and translating it off-screen, leading to memory issues and synchronization problems.

Technical Anatomy

We achieved this custom navigation model that operates based on the drawable navigation, for secondary actions and the bottom navigation for primary actions as such:

Centralized Initialization: The `setupDrawer` method is called only once upon user login and uses the `userViewModel` to determine the user's role. Based on the role, it dynamically loads the appropriate XML layout into the drawer container. This corrected our previous structural error by ensuring the drawer only exists once within the main activity's root layout (Android Developers, 2025; Vogella, 2023). This efficient architecture was adapted based on the Singleton pattern: only one navigation drawer for every screen (Gamma et al., 1994).

Efficient Menu Trigger: We intentionally placed the menu trigger on the `BottomNavigationView`. The `setOnMenuItemClickListener` on this specific item is used only to invoke `drawerLayout.openDrawer()`. This separation of

concerns ensures the menu button doesn't attempt to load a new fragment, which would be inefficient.

Role-Based Fragment Mapping: The core complexity is found in NavigationItemSelected. This method acts as the main routing logic for the entire application. It first checks the user's role using `userViewModel.isPoolOwner()` or `userViewModel.isServiceProvider()`. It then uses nested if/else if blocks to map the selected menu item ID to the correct, role-specific Fragment instance via the `selectedFragment`, a custom variable. This simplistic structure was then complicated by passed of data for various instances of navigation, such as navigating from the pool owner home screen to the pool pH calculator screen. We reduced the complexity by using shared models, class that store general public data and then triggering the appropriate functionality based on the screen being called. This guarantees that each user role is contained within its own designated application experience. This good practice is accredited to Liskov & Zilles (1974) for efficient abstraction of data types, allowing us to maintain data integrity throughout SplashScreen (Liskov & Zilles, 1974)

1.2.3 Custom toast notification

Extract: A section of code in the `NotificationHelper` class that shows custom notifications

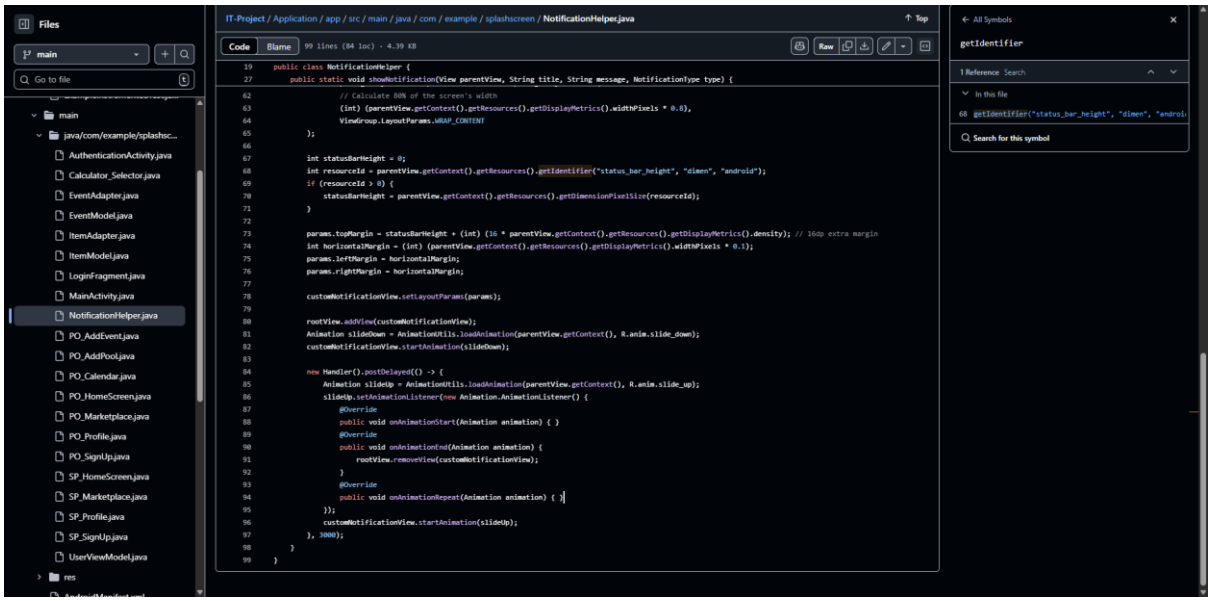

Figure 1.5 A code snippet for SplashScreen's notifier

Theoretical Anatomy

To provide a consistent, branded, and non-intrusive feedback mechanism, we chose to replace the standard Android `Toast` notifications with a custom, ribbon-style UI component. This required writing a dedicated helper class, `NotificationHelper.java`, to manage the entire display lifecycle, from injection into the main view hierarchy to timed removal. The design uses an **`Enum`** (`NotificationType`) to handle different instances such as `CONFIRMATION`, `ERROR`, or `INFORMATIONAL`, allowing the system to automatically select the correct styling for the message being displayed.

Technical Anatomy

Dynamic Positioning and Sizing: Unlike standard UI elements, this notification is added directly to the root view of the activity `(rootView.addView(customNotificationView))` which is a bit more complex to achieve (Chen, 2023). To center and size it correctly on any device, we had to calculate its dimensions dynamically:

Width: Hardcoded to 80% of the total screen width (`widthPixels` * 0.8).

Margins: The remaining 20% is split equally for the left and right margins (`widthPixels` * 0.1), ensuring the notification is perfectly centered.

Top Margin: To ensure the notification slides down below the device's status bar, we dynamically calculate the height of the status bar and add a small offset ,16dp, converted to pixels using density for padding. This crucial step prevents the notification from being hidden behind system elements.

Timed Animation Lifecycle: The component's appearance and disappearance are managed entirely by custom XML animations (`slide_down` and `slide_up`). A `Handler().post Delayed()` function is used to control the duration, which is 3000ms, and a custom Animation Listener is implemented. The listener is critical because it ensures the view is only removed from the rootView in the `onAnimationEnd` callback. Removing the view before the animation finishes would result in an abrupt disappearance and potential crashes. This provides a smooth, reliable transition that mimics native system notifications.

1.3 Source code references

This section compiled by ***The Ripple Effect***, is a declarative one in nature. We acknowledge and give attribution to the respective entities that laid the technical foundation for the development of *SplashScreen*. The format of each source code extract has been adapted from the template.

1.3.1 Implementing a Fragment-to-Fragment Communication Pattern

Description

In modern Android development, the legacy practice of communicating between Fragments using interfaces implemented by the parent Activity is replaced by the official FragmentResultListener API. This mechanism, used to pass data such as confirmation flags or IDs between the `PO_AddEvent` and `PO_Calendar` Fragments, is required for reliable, lifecycle-aware communication (Android Developers, 2025).

Usage

PO_Calendar.java (setupEventResultListener() method) and PO_AddEvent.java

1.3.2 Constructing Time-Based Range Queries in Cloud Firestore

Description

To efficiently filter and display maintenance events for a single day, the application uses composite Firestore queries. Specifically, the use of `whereGreaterThanOrEqualTo()` and `whereLessThanOrEqualTo()` on the `startDate` field, combined with the orderBy() clause, is an advanced querying pattern essential for scheduling systems. This requires precise handling of `System.currentTimeMillis()` timestamps and the creation of specific backend indexes to run successfully and maintain performance (Android Developers, 2025; Firebase Developers, 202).

Usage

`PO_Calendar.java` (`loadEventsForSelectedDate()` method)

1.4 Problems encountered

This section provides a reflection on the technical and logistical difficulties encountered during the project's development, outlining the challenges and the eventual strategies we employed to overcome them.

1.4.1 Integrating a Functional and Customizable Calendar System

Problem

We encountered significant difficulty in establishing a calendar component that was both highly functional for scheduling and fully customizable to fit the SplashScreen user interface. Our initial approach involved evaluating several third-party Android libraries and packages. While some libraries offered a beautiful calendar view, they often imposed rigid internal structures that made it nearly impossible to integrate our specific Firebase event-fetching logic or customize the event display, e.g., our colour-coding style, custom event titles, without heavily forking the library code.

Solution

We resolved this by abandoning external libraries and leveraging the built-in Android CalendarView component. This component provided the basic month view. Crucially, we custom-wrote all the related logic, including the time manipulation, database querying, as seen in Section 1.2, and the display of the events list, `RecyclerView`. This allowed us to maintain full control over the UI/UX and integrate our complex data architecture seamlessly.

1.4.2 Managing Git Version Control and Merge Conflicts

Problem

Throughout the development phase, particularly when transitioning between personal and campus workstations, we frequently encountered Git merge conflicts. These were usually due to us forgetting to pull the latest changes or commit local modifications before switching work environments. While our system utilized separate development and production branches, human error caused these inconsistencies in the local environment, leading to frustrating conflicts upon attempting a final push or merge.

Solution

To resolve these issues efficiently, we relied heavily on GitHub desktop's history tool. We would often use the logs and check the differences to precisely identify the conflicting changes (GitHub Docs, 2025). The common resolution involved checking out specific commits, comparing the applicable code fragments manually, and then compiling the required changes into a single, converged version. This rigorous manual process, while time-consuming, ensured no critical logic was accidentally discarded.

1.4.3 Data Dependency and Implementation of Offline Capabilities

Problem

A persistent challenge has been managing the application's reliance on a stable internet connection. While Firebase is strong, users need to view recent pool logs or maintenance schedules even when temporarily offline. Achieving a complete, reliable offline-first state is complex due to the asynchronous nature of Fibebase Firestore's data synchronization.

Solution

Our current strategy focuses on managing cache dependency. We utilized Firestore's built-in offline capabilities, which allow the app to cache data it has recently accessed while online (Firebase Developers, 2025). This provides limited offline functionality. For future iterations, the goal is to implement a more comprehensive cache-management layer potentially using the Repository pattern to actively store and serve data like event logs during offline periods, with synchronization only occurring when connectivity is restored (Gamma et al., 1994).

1.4.4 Storing User Images and Overcoming Firebase Storage Limits

Problem

We faced a logistical hurdle regarding the storage of large binary data, specifically user-uploaded pool images, necessary for both pool owners and service providers. While Cloud Firestore is ideal for transactional data, it is not designed to store images. Images must be stored in Firebase Cloud Storage Buckets. The free tier of Firebase Storage imposes rate limits and bandwidth restrictions, which, combined with the administrative requirement of

linking a credit card for initial configuration even for the free tier, presented a barrier since the project is currently unfunded.

Solution

We devised a conceptual workaround involving an external cloud provider, Cloudinary, to handle the image hosting (Cloudinary Documentation, 2025). The conceptual flow involves:

1. The user uploads a pool image from the app.
2. The app sends the image directly to Cloudinary.
3. Cloudinary returns a secure URL.
4. This URL, not the image itself, is stored in the corresponding pool document in Cloud Firestore.
5. When a user, owner or service provider, views the pool, the app fetches the URL from Firestore and downloads the image from Cloudinary. This decentralized approach allows us to manage image storage and delivery independently of Firebase's immediate financial constraints.

1.5 References

Android Developers. (2025). Pass results between fragments. Retrieved September 14, 2025, from https://developer.android.com/guide/fragments/pass-data

Android Developers. (2025). Simple and compound queries in Cloud Firestore. Retrieved September 23, 2025, from https://firebase.google.com/docs/firestore/query-data/queries

Android Developers. (2025). *Build responsive UI with ConstraintLayout*. Retrieved October 10, 2025, from https://developer.android.com/develop/ui/views/layout/constraint-layout

Chen, B. (2023). *Understanding the Android View Hierarchy and Drawing*. Retrieved September 18, 2025, from https://medium.com/androiddevelopers/understanding-the-android-view-hierarchy

Cloudinary Documentation. (2025). *Cloudinary SDK for Android*. Retrieved October 09, 2025, from https://cloudinary.com/documentation/android

Firebase Developers. (2025). *Add Firebase to your Android project*. Retrieved September 25, 2025, from https://firebase.google.com/docs/android/setup

Firebase Developers. (2025). *Secure your data in Cloud Firestore*. Retrieved September 26, 2025, from https://firebase.google.com/docs/firestore/security/overview

Gamma, E., Helm, R., Johnson, R., & Vlissides, J**.** (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

GitHub Docs. (2025). *Resolving a merge conflict using the command line*. Retrieved September 21, 2025, from https://docs.github.com/articles/resolving-a-merge-conflict-using-the-command-line

Liskov, B. H., & Zilles, S. (1974). Programming with abstract data types. *ACM Sigplan Notices*, *9*(4), 50–59.

UiLover. (n.d.). *Android UI Lover* [YouTube channel]. Retrieved September 14, 2025, from https://www.youtube.com/@UiLover

Vogella, L. (2023). *Android Development Tutorial: Fragments*. Retrieved September 23, 2025, from https://www.vogella.com/tutorials/AndroidFragments/article.html